

第 1 章 变量和基本类型

环境装好了，这一章开始真正写代码。我们做一个小程序，输入几门课的分数，算出平均分并给个等级。麻雀虽小，变量、数字、文字、判断输出这些最基本的东西都会用到。

1.1 先算一份成绩单

本节任务：写一个程序，把三门课的分数存起来，算出总分和平均分，再按平均分给一个等级，最后打印成一份好看的成绩单。

新建一个文件叫 score.py，写入下面的代码：

```
1 name = "小明"
2 chinese = 86
3 math = 92
4 english = 78
5
6 total = chinese + math + english
7 average = total / 3
8
9 print("姓名: ", name)
10 print("总分: ", total)
11 print("平均分: ", average)
```

运行 python score.py，输出是：

```
姓名:  小明
总分:  256
平均分:  85.33333333333333
```

能算了，但平均分那一长串小数很难看。而且我们还想按分数给个等级。把程序改成下面这样：

```
1 name = "小明"
2 chinese, math, english = 86, 92, 78 # 一行赋三个值
3
4 total = chinese + math + english
5 average = total / 3
6
7 if average >= 90:
8     level = "优秀"
9 elif average >= 80:
```

```
10     level = "良好"
11 else:
12     level = "还需努力"
13
14 print(f"{name} 的成绩单")
15 print(f"语文 {chinese} 数学 {math} 英语 {english}")
16 print(f"总分 {total}, 平均分 {average:.1f}, 等级 {level}")
```

这回输出干净多了：

```
小明 的成绩单
语文 86 数学 92 英语 78
总分 256, 平均分 85.3, 等级 良好
```

短短十几行里出现了好几样新东西，变量、两种数字、文字、条件判断、还有那个能把数字塞进文字里的写法。下面逐个讲清楚。

1.2 变量就是给数据起个名字

本节任务：搞懂等号在编程里的含义，以及名字该怎么取。

上面的 `name = "小明"` 里，`name` 是**变量**。变量就是给一份数据起个名字，之后用 **★核心** 这个名字就能取到数据。等号在这里不是数学里的相等，而是**赋值**，意思是把右边的东西放进左边这个名字里。所以下面这行在数学上说不通，在编程里却很常见：

```
1 count = 5
2 count = count + 1 # 先算右边 5+1 得 6, 再放回 count
3 print(count) # 输出 6
```

变量可以随时改成别的值，也可以改成别的类型的值。取名字有几条规矩，必须遵守的是只能用字母、数字和下划线，不能以数字开头，也不能用 Python 已经占用的词比如 `if` 和 `for`。

约定俗成但同样重要的是，名字要能说明这是什么。写 `a` 和 `b` 当时省事，过两天就不知道存的是什么了。多个单词之间用下划线连起来，比如 `total_score` 比 `totalscore` 好读。

1.3 数字有两种

本节任务：分清整数和小数，知道除法为什么总得到小数。

Python 里常用的数字有两种。**整数**就是不带小数点的数，比如 86 和 -3。**浮点数**是带小数点的数，比如 85.3 和 3.14，浮点是小数在计算机里的存法。

运算符和数学课上基本一样，加减乘除写作加号、减号、星号、斜杠。有三个要单独说：

```
1 print(7 / 2) # 2 除法，结果总是小数 3.5
2 print(7 // 2) # 2 整除，只要整数部分 3
3 print(7 % 2) # 2 取余，余数 1
4 print(2 ** 10) # 2 乘方 1024
```

注意 **除法结果总是小数**，即使能除尽也一样， $8 / 2$ 得到的是 4.0 而不是 4。想要整数就用两个斜杠的整除。取余在判断奇偶时很好用，一个数对 2 取余是 0 就是偶数。★核心

浮点数还有一个容易惊讶的地方：

```
1 print(0.1 + 0.2) # 输出 0.30000000000000004
```

这不是 Python 的毛病，而是小数在二进制里存不精确，几乎所有语言都这样。日常使用不用管，只要记住不要直接判断两个小数是否相等，需要精确计算金额时另有专门的办法。

1.4 文字叫字符串

本节任务：会写文字、拼接文字，并把数字漂亮地嵌进文字里。

用引号括起来的文字叫**字符串**。单引号双引号都可以，只要成对。

```
1 a = "小明"
2 b = '你好'
3 c = "他说：'走吧'" # 里外用不同的引号就不会打架
```

字符串能用加号拼起来，用星号重复：

```
1 print("你" + "好") # 你好
2 print("-" * 20) # 打印 20 个短横线，用来分隔很方便
```

但拼接有个麻烦，加号两边必须都是字符串，混进数字就会报错。所以更好的办法是**格式化字符串**，也就是开头那个 f 打头的写法。在引号前面加一个 f，然后把变量写进大括号里：★核心

```
1 name, score = "小明", 85.333
2 print(f"{name} 考了 {score} 分") # 小明 考了 85.333 分
3 print(f"平均分 {score:.1f}") # 平均分 85.3
4 print(f"平均分 {score:.2f}") # 平均分 85.33
```

大括号里冒号后面是格式说明，点一 f 表示保留一位小数，点二 f 就是两位。开头

那份成绩单里的 `average:.1f` 就是这么把长长的小数变短的。这个写法后面会一直用。

1.5 真假和判断

本节任务：认识只有两个取值的布尔类型，并用它做条件判断。

比较两个东西会得到**布尔值**，它只有两个取值，`True` 表示真，`False` 表示假，注意首字母大写。

```
1 print(85 >= 90) # False
2 print(85 >= 80) # True
3 print(3 == 3) # True 两个等号才是判断相等
4 print(3 != 4) # True 不等于
```

这里有个新手必踩的坑，**一个等号是赋值，两个等号才是判断相等**。写成一个等号 ★核心轻则结果不对，重则直接报错。

有了布尔值就能做判断。开头程序里的写法是这样的：

```
1 if average >= 90:
2     level = "优秀"
3 elif average >= 80:
4     level = "良好"
5 else:
6     level = "还需努力"
```

`if` 后面跟条件再跟一个冒号，下一行缩进四格写条件成立时做的事。`elif` 是否则如果，可以有很多个，`else` 是以上都不成立时做的事。Python 从上往下检查，哪个条件先成立就执行哪一块，剩下的都跳过。所以 85.3 会在第二个条件停下，得到良好。

分支还有更多用法，下一章会专门展开，这里够用了。

1.6 类型不对就会出错

本节任务：知道怎么查看和转换类型，避开最常见的一类报错。

每个数据都有类型，用 `type` 可以查：

```
1 print(type(86)) # <class 'int'> 整数
2 print(type(85.3)) # <class 'float'> 浮点数
3 print(type("小明")) # <class 'str'> 字符串
4 print(type(True)) # <class 'bool'> 布尔
```

类型不对的操作会报错，比如字符串加数字：

```
>>> "分数" + 86
TypeError: can only concatenate str (not "int") to str
```

TypeError 就是类型不匹配。解决办法是转换，用 int 转整数，float 转小数，str 转字符串：

```
1 print("分数" + str(86)) # 分数86
2 print(int("86") + 1) # 87
3 print(float("3.5") + 1) # 4.5
4 print(int(3.9)) # 3 直接砍掉小数，不是四舍五入
```

转换在读用户输入时特别重要。**input 拿到的永远是字符串**，哪怕用户输的是数字：★核心

```
1 age = input("请输入年龄：") # 用户输入 18
2 print(age + 1) # 报错，字符串不能加数字
3 print(int(age) + 1) # 19 转成整数才能算
```

这是新手最常撞的一堵墙，看到 TypeError 先想想是不是忘了转换。

1.7 小结

这一章写出了第一个有用的程序。变量是给数据起的名字，等号是赋值不是相等。数字分整数和浮点数，单斜杠除法总得小数，双斜杠整除，百分号取余。文字叫字符串，用 f 打头的格式化写法能把变量和格式一起嵌进去。比较得到布尔值，两个等号才是判断相等。类型不对会报 TypeError，input 拿到的永远是字符串，要算数得先转换。下一章开始处理成批的数据。

本章知识点

- **变量**——给数据起的名字，等号是把右边的值放进左边这个名字
- **整数与浮点数**——不带小数点的和带小数点的，浮点是小数在计算机里的存法
- **除法**——单斜杠结果总是小数，双斜杠只取整数部分，百分号取余数
- **字符串**——引号括起来的文字，单双引号都行，只要成对
- **格式化字符串**——引号前加 f，把变量写进大括号，冒号后可指定保留几位小数
- **布尔值**——只有 True 和 False 两个取值，比较运算的结果
- **一个等号与两个等号**——一个是赋值，两个才是判断相等，新手必踩的坑
- **type**——查看一个数据是什么类型
- **类型转换**——int 转整数，float 转小数，str 转字符串
- **TypeError**——类型不匹配，比如字符串直接加数字
- **input**——读用户输入，拿到的永远是字符串，要算数必须先转换